COMP0222 Simultaneous Localisation and Mapping

Lecture 10: A Shallowish Dive into COLMAP

Simon Julier

s.julier@ucl.ac.uk

Sajad Saeedi

s.saeedi@ucl.ac.uk

Motivation

- We've seen the basic estimation problem behind SLAM
- We've seen how, for a camera, you can obtain features and you can match them to form a set of observations
- We are now going to look at a couple of successful, widely-used systems: COLMAP and ORB-SLAM2
- A joint lecture on both became too large, so we are going to look at COLMAP and ORB-SLAM2 separately

Structure

- Overview of Both Systems
- Common Design Principles
- An Overview of COLMAP

Overview of Both Systems

- Overview of Both Systems
- *Common Design Principles*
- *An Overview of COLMAP*

COLMAP

- COLMAP is a system for Structure-from-Motion (SfM)
- The goal is to produce accurate and complete models of the environment from *image collection*
- The images can come in any order from lots of people at lots of different times using lots of different cameras
- The quality of the map is the priority – the estimated camera location is a “nuisance” parameter
- Real-time behaviour is not required, but it should be *usably fast*

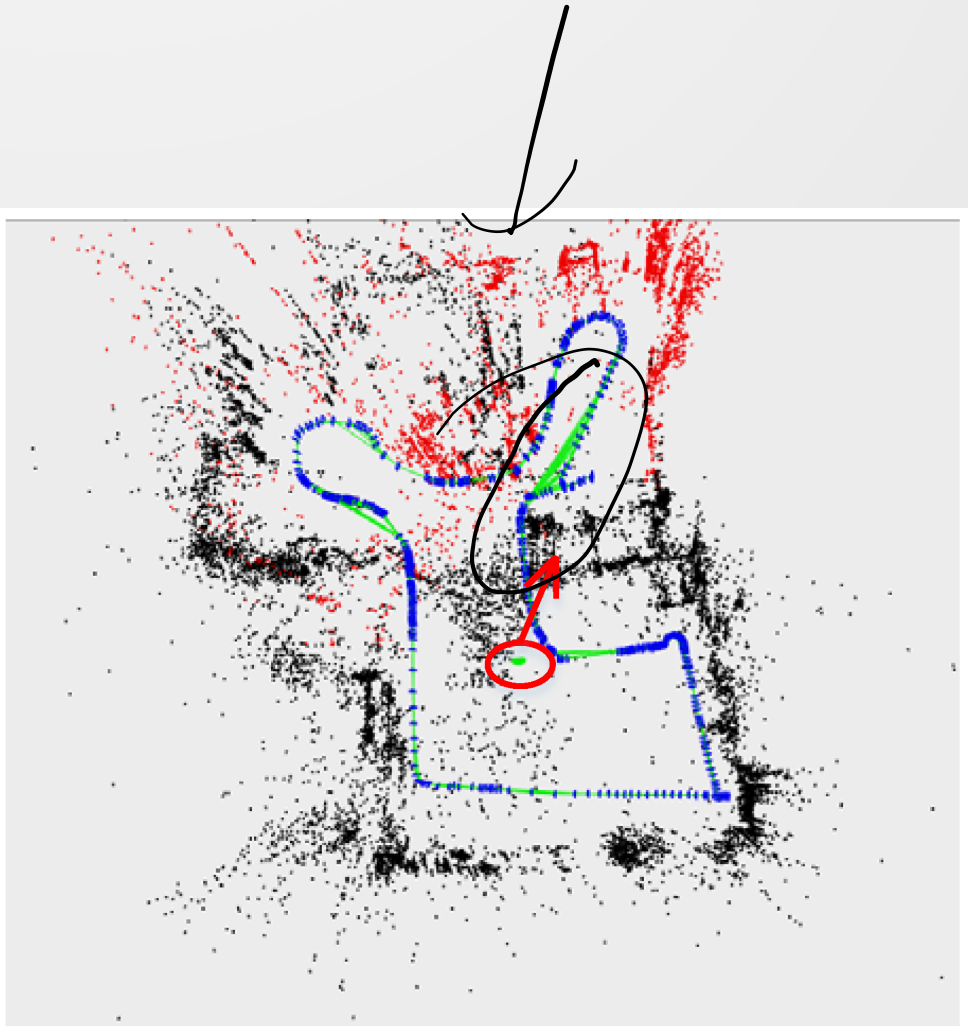
COLMAP

ORB-SLAM2

- ORB-SLAM2 is a camera-based SLAM system
- The goal is to produce an estimate of the pose (position, orientation)
- The same camera is used all the time and progresses along a smooth trajectory
- The accuracy of the camera is priority – the estimated map is a "nuisance" parameter
- *Real-time behaviour is required*

ORB-SLAM2

(a)



(b)

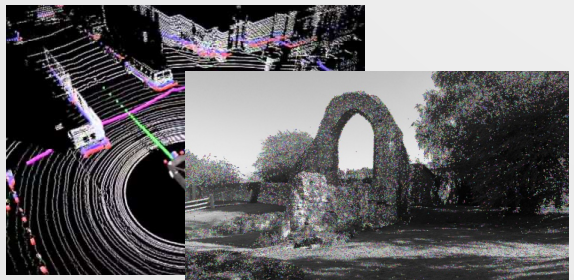
Common Design Principles

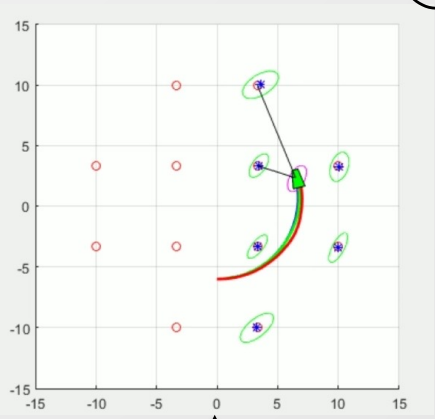
- *Overview of Both Systems*
- Common Design Principles
- *An Overview of COLMAP*

Common Aspects of Both Systems

- **Despite their very different design goals, COLMAP and ORB-SLAM2 have similarities in design and architecture:**
  - **Both are broadly architected with a front-end / back-end architecture**
  - **Both use similar representations of the camera and the landmarks**

Front-End / Back-End Architecture





Frontend

Feature Map

Subset of “interesting” information

Backend

Front-End / Back-End Architecture

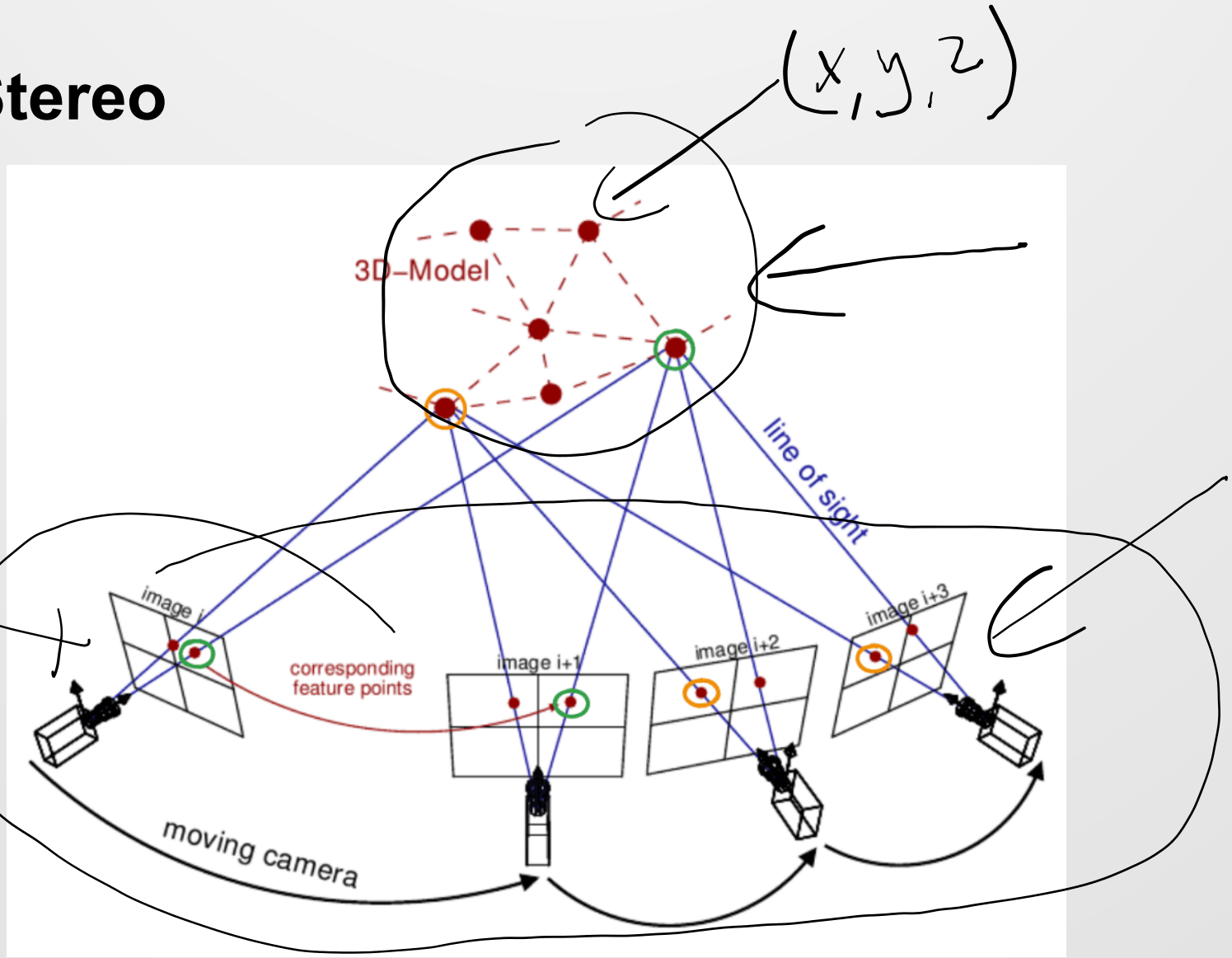
- The front-end tries to do as much processing of the image data as possible
- This includes frame-to-frame data association and place recognition
- The back-end does estimation using factor graphs and optimization-based approaches
- There are numerous tweaks and refinements to this basic layout though

Camera and Landmark Representations

- Both COLMAP and ORB-SLAM2 represent the system using ideas from multi-view stereo

- In this approach, we have a set of cameras and a set of landmarks

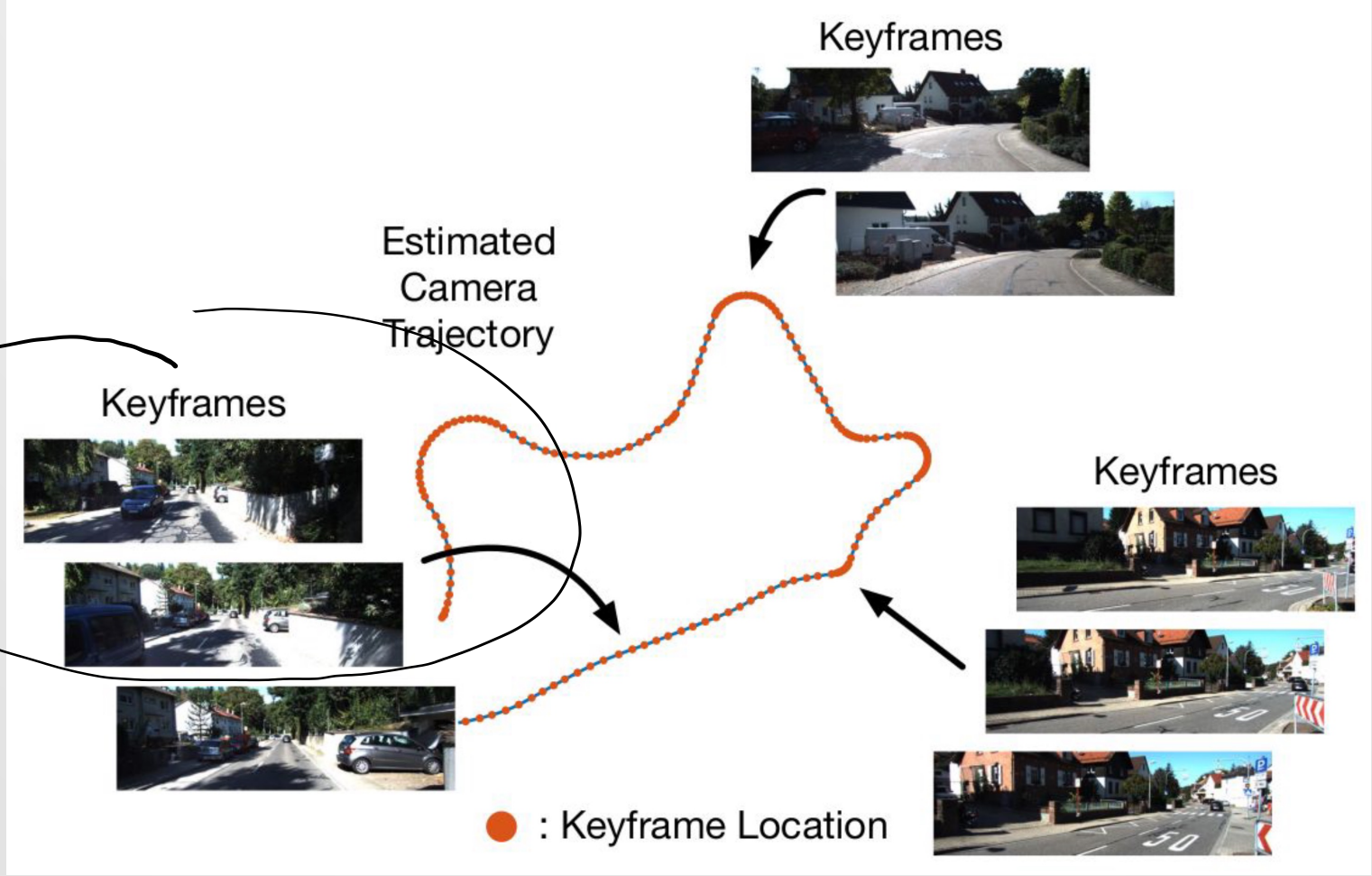
Multi-View Stereo



Camera Poses and Keyframes

- Each camera has a vertex which represents its state
- Associated with each vertex is an image called a *keyframe*
- There are *no* process model edges
  - For COLMAP the image collection can enter the system in an arbitrary order
  - For ORB-SLAM2, it massively simplifies the system (if you have a lot of measurements)

Keyframes

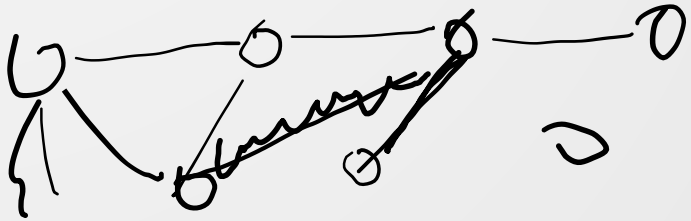

Keyframes
Estimated
Camera
Trajectory
Keyframes
Keyframes
: Keyframe Location

Landmarks

- Each landmark is a vertex which has its own 3D state information

- Each vertex has, associated with it, a feature descriptor which can be used to look the feature up

Establishing Observations

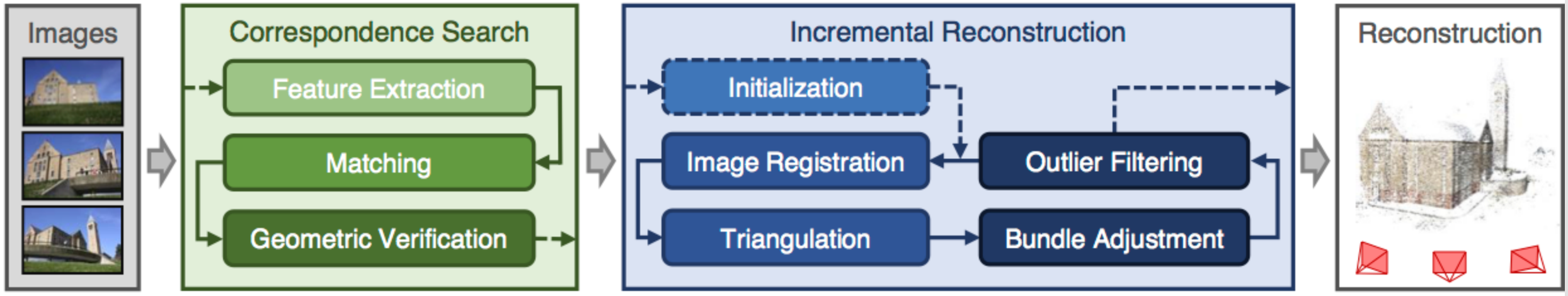


- Observations are established through data association:
  - The frame associated with the camera vertex, and potential landmark features are detected and described
  - Appearance-based matching and geometric-consistency are used to match detected features with the matched points
  - Observations are created from successful matches
- Both COLMAP and ORB-SLAM2 have ways to “back out” associations if they are wrong

An Overview of COLMAP

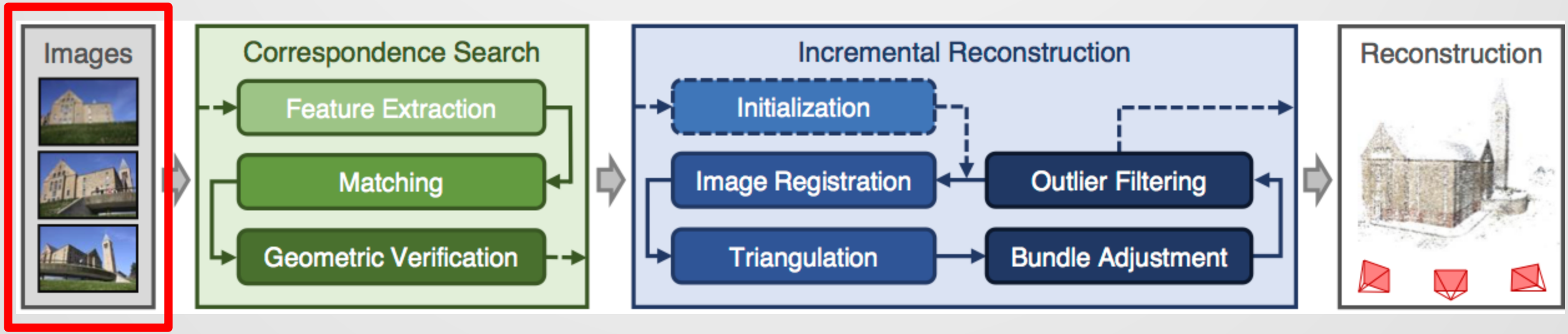
- *Overview of Both Systems*
- *Common Design Principles*
- An Overview of COLMAP

High-Level Architecture of COLMAP



COLMAP's incremental Structure-from-Motion pipeline

Images



COLMAP's incremental Structure-from-Motion pipeline

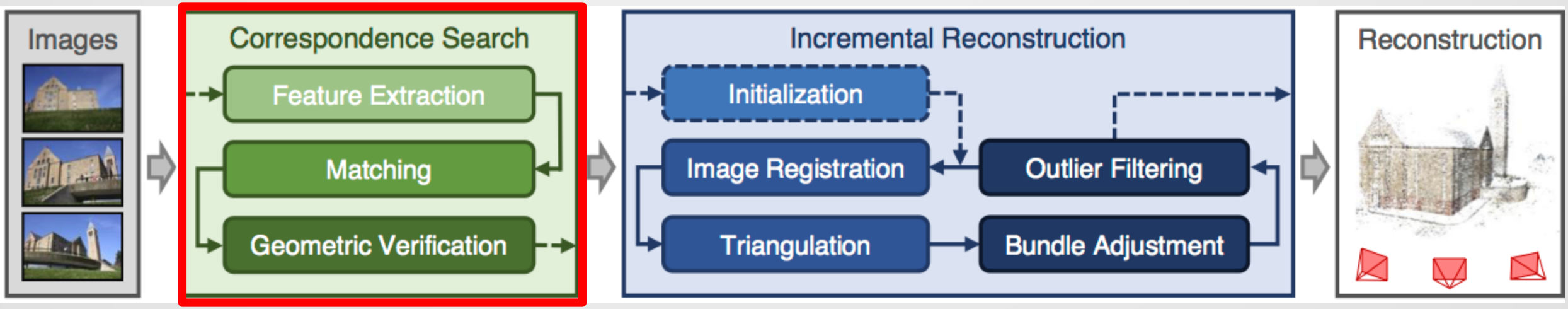
Images

- **COLMAP is provided with an input set of images:**

$$\mathcal{I} = \{I_i | i = 1, \ldots, N_I\}$$

- **These are all provided in advance**

Correspondence Search



[COLMAP's incremental Structure-from-Motion pipeline]

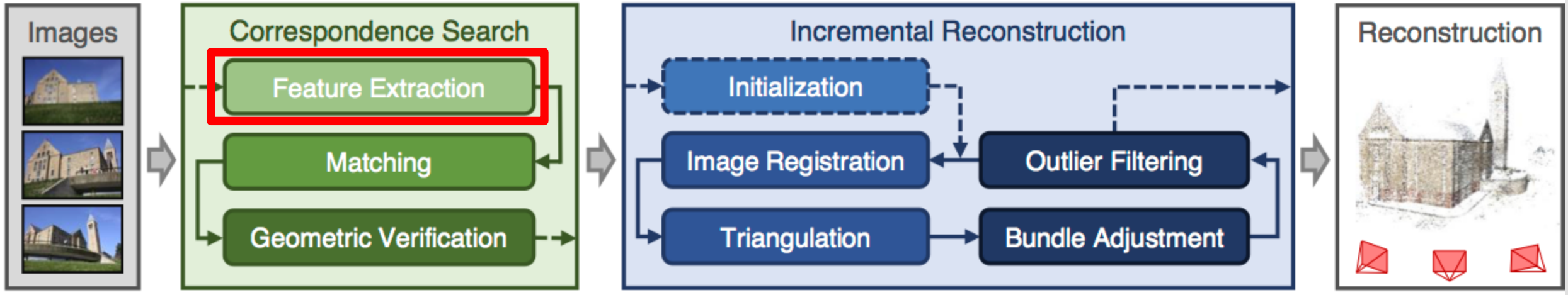
Correspondence Search

- In this stage, all the raw frames are processed to identify features, matches and to build up a candidate set of measurements

- The output from this stage is encoded in a *scenegraph*

↳ pairwise

Feature Extraction



COLMAP's incremental Structure-from-Motion pipeline

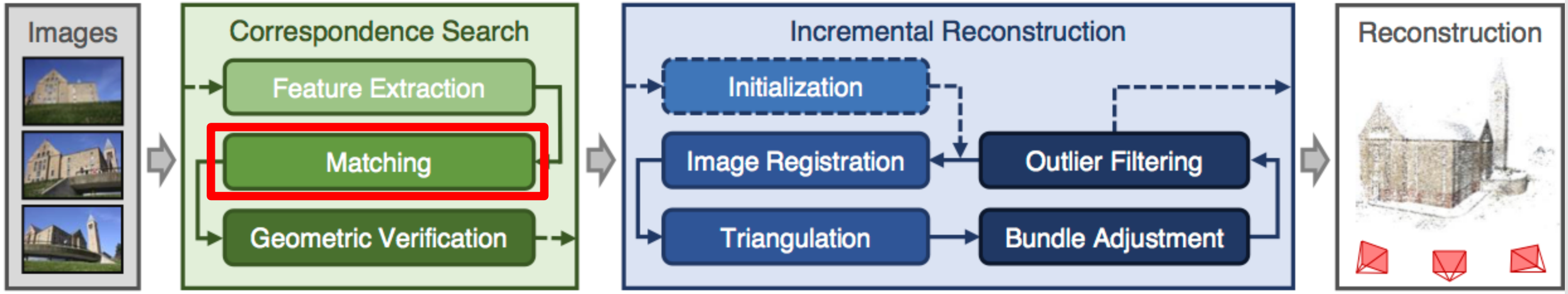
Feature Extraction

- COLMAP iteratively goes through all the images and the extracts features

$$\mathcal{F}_i = \{(\mathbf{x}_j | \mathbf{f}_j) \,|\, j = 1, \ldots, N_{F_i}\}$$

- SIFT is commonly used, but the interface is pluggable and other descriptors such as SuperPoint can be used

(Feature-Based) Matching



COLMAP's incremental Structure-from-Motion pipeline

(Feature-Based) Matching

- Now the features have been extracted, we need to look for potentially overlapping frames with common features
- This output is the set of overlapping image pairs and feature correspondences

$$\mathcal{C} = \{(I_a, I_b) \,|\, I_a, I_b \in \mathcal{I}, a < b\}$$

$$\mathcal{M}_{ab} = \mathcal{F}_a \times \mathcal{F}_b$$

Importance of the Matching Strategy

Matching Strategies

- **Exhaustive Matching**

Exhaustive Matching

Exhausting Matching

- Exhaustive matching checks if every frame matches with every other frame in the dataset
- It is the most comprehensive matching strategy
- However, it is computationally expensive

$$O\left(N_I^2 N_{F_i}^2\right)$$

- It can also be prone to false positives

Matching Strategies

- **Exhaustive Matching**
- **Sequential Matching**

Sequential Matching

- Suppose we know the images come from, say, an uninterrupted video
- This introduces spatial and temporary coherency
- Therefore, the idea is to match current frame with, say, 10 frames before and 10 frames after based on time

Sequential Matching

Sequential Matching

- It can be significantly faster
- However, it behaves like an odometry system which means it will exhibit drift
- Therefore, it has to use appearance-based recognition to effectively do loop closure
- Also, the model will be sparser and so less accurate

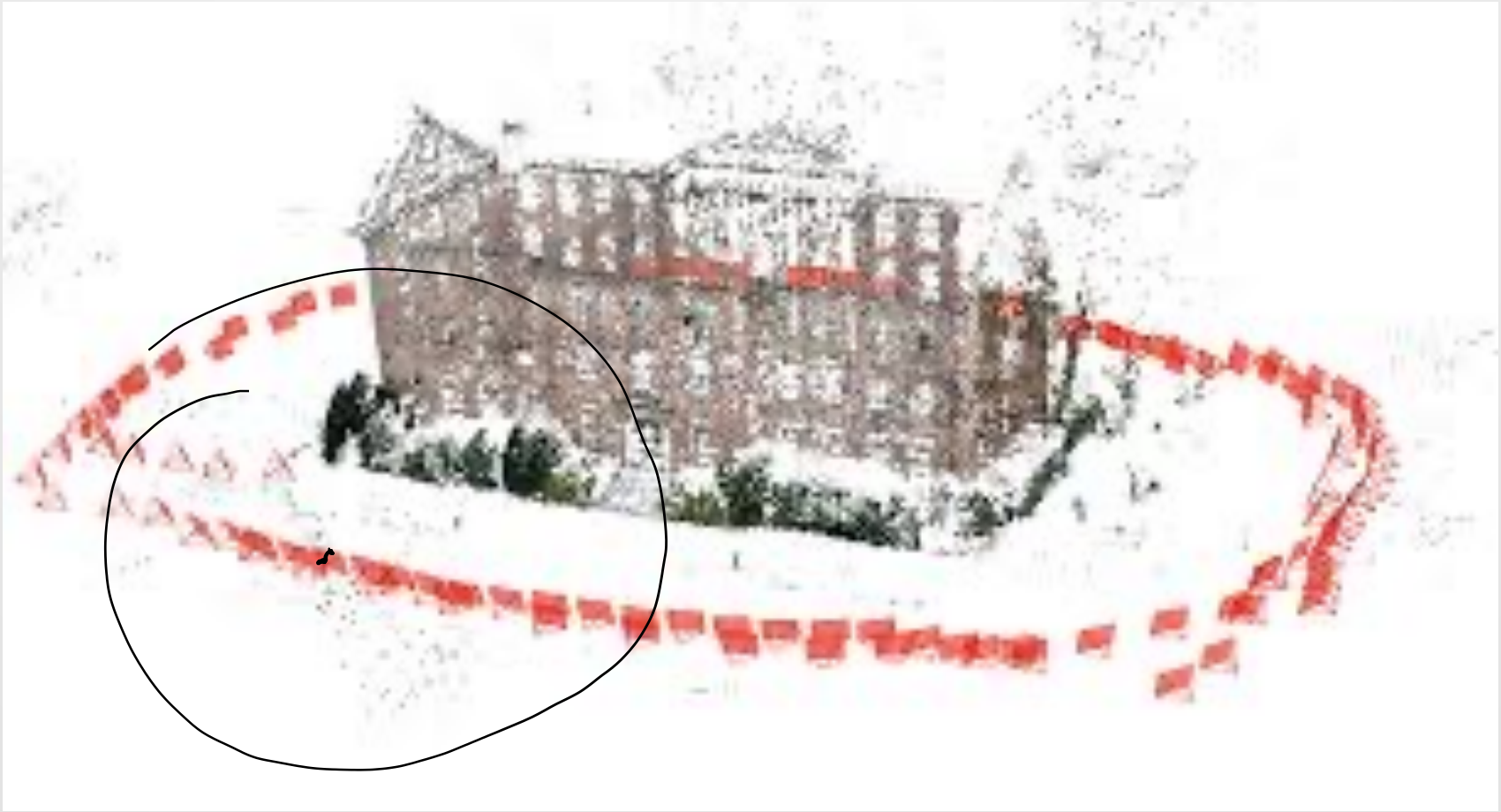
Matching Strategies

- Exhaustive Matching
- Sequential Matching
- Spatial Matching

Spatial Matching

- Many cameras can write out location information (e.g., from GNSS) into EXIF tags
- COLMAP can read these
- The idea is that images taken near to one another are likely to associate
- COLMAP will only match images within a set distance of one another

Spatial Matching



Matching Strategies

- Exhaustive Matching
- Sequential Matching
- Spatial Matching
- Vocabulary Tree Matching

Vocabulary Tree Matching

- This is a variant of the appearance-based matching discussed last week
- Images are classified with coarse image descriptors
- Precise geometric alignment is carried out later
- Pre-trained dictionaries exist
- COLMAP includes code to train your own dictionary

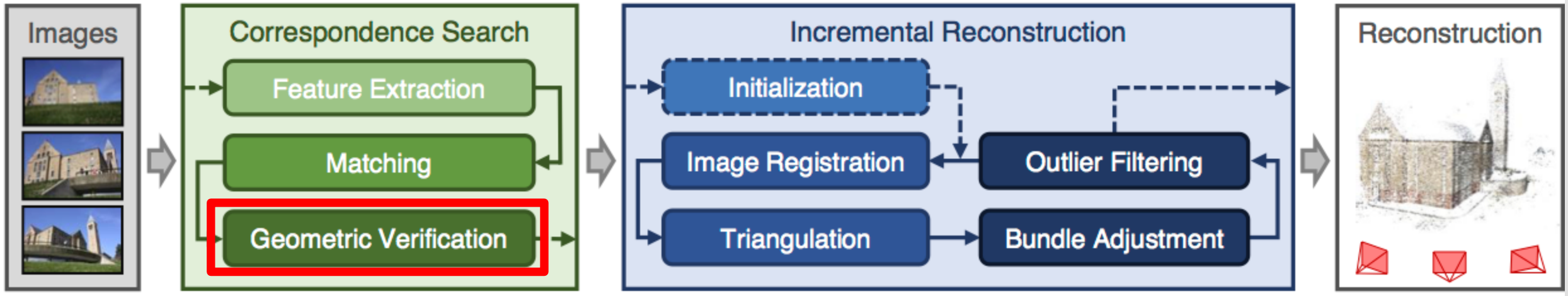
Matching Strategies

- Exhaustive Matching
- Sequential Matching
- Spatial Matching
- Vocabulary Tree Matching
- Transitive Matching
- Custom Matching

Transitive and Custom Matching

- **Transitive Matching:**
  - This attempts to extend identified by matches by exploring transitive relationships
  - If A matches with B, and B matches with C, then try to match A and C
- **Custom Matching:**
  - Provide an external list of matches
  - Can use self-identifying markers

High-Level Architecture of COLMAP



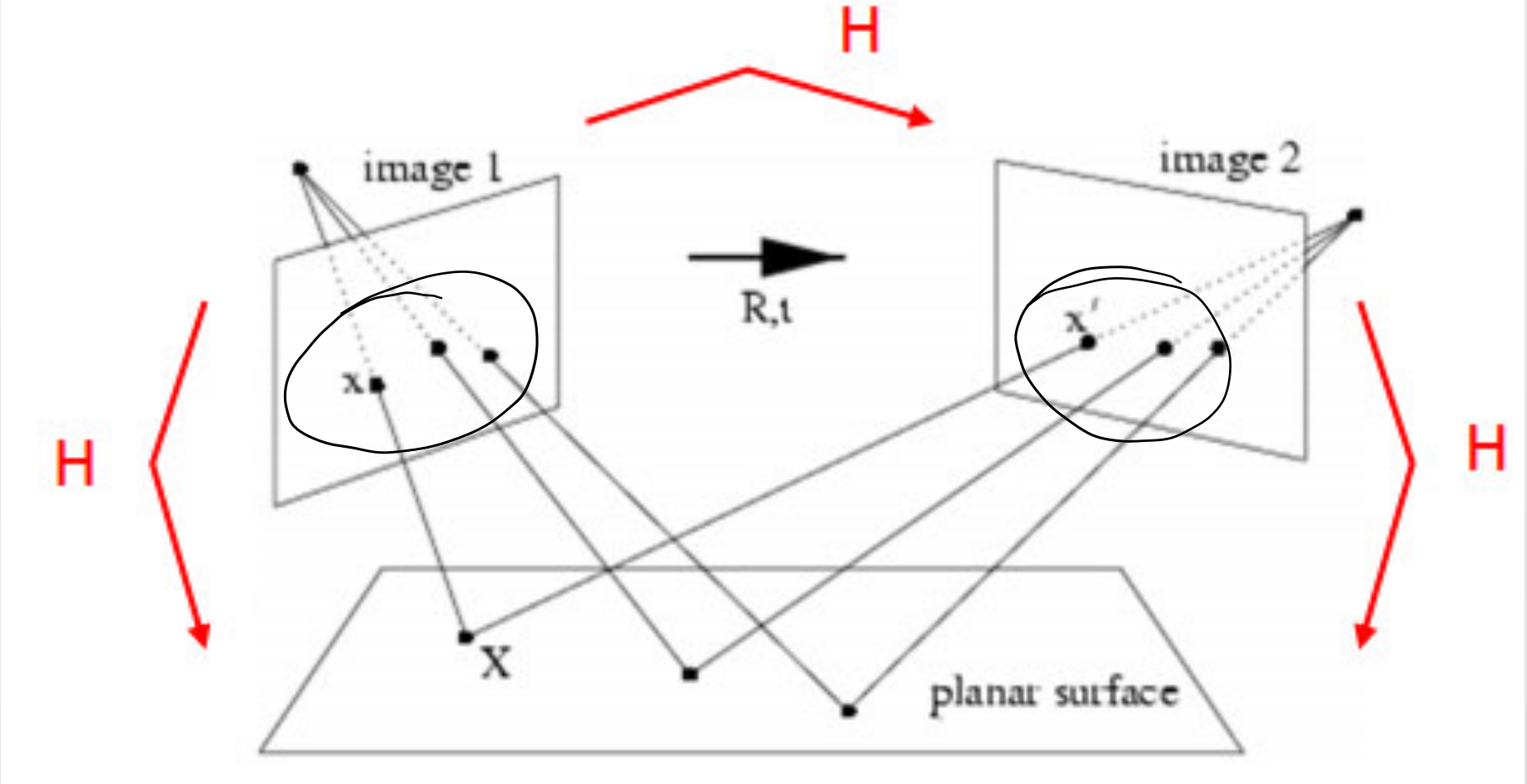
COLMAP's incremental Structure-from-Motion pipeline

Geometric Verification

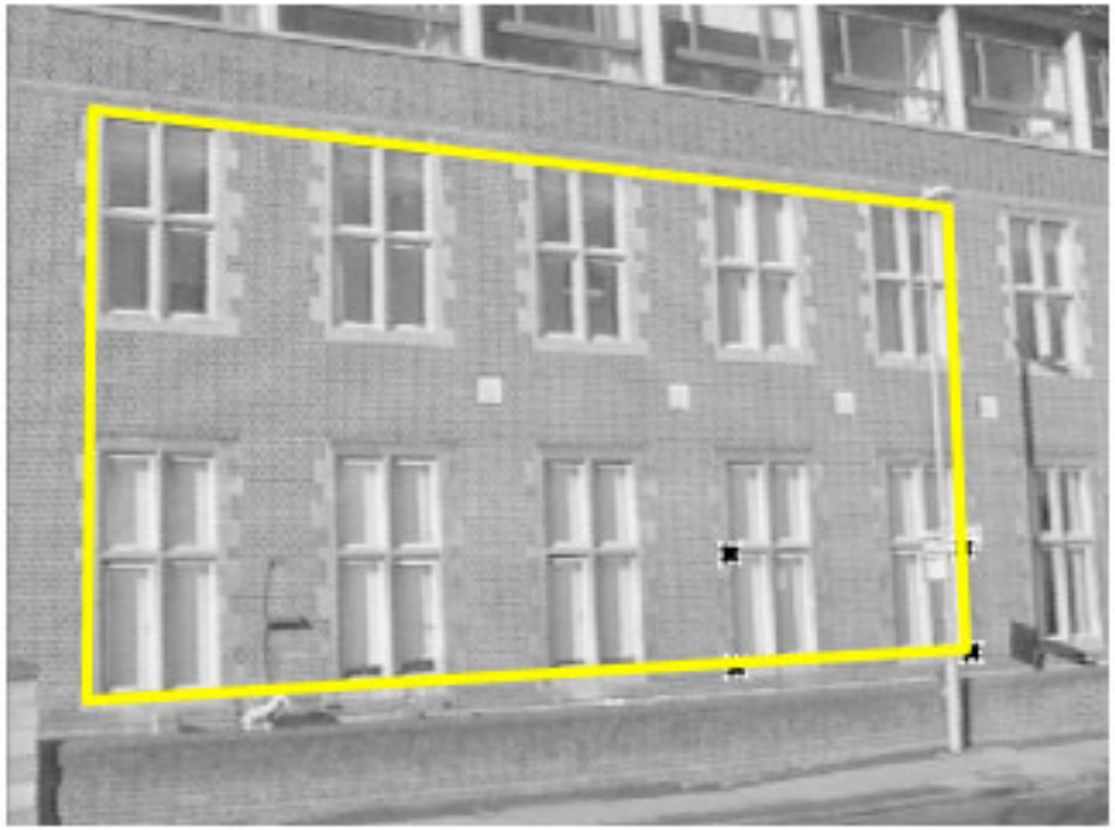
- As we saw last week, pure feature-based matching can have matching errors
- Therefore, we use RANSAC to estimate the feature matches
- Since we don't know the geometry of the scene, three models can be used:
  - Homographies
  - Fundamental matrices
  - Essential matrices

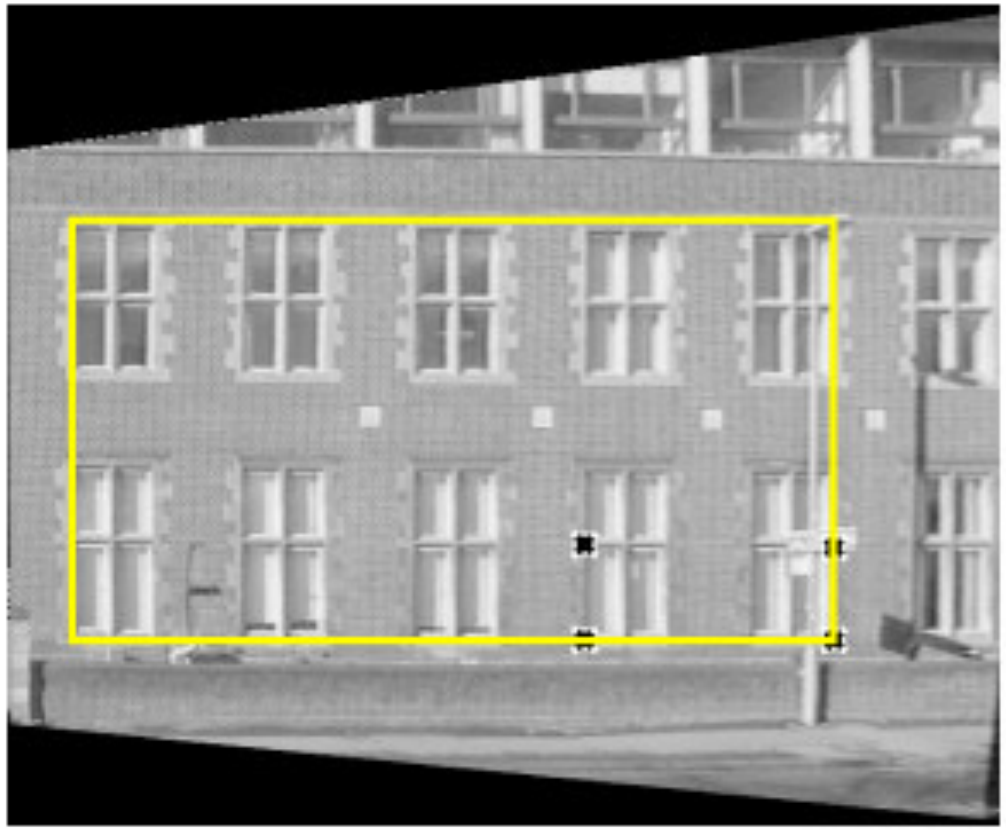
Homographies

H f_x f_y



Homographies in the Real World



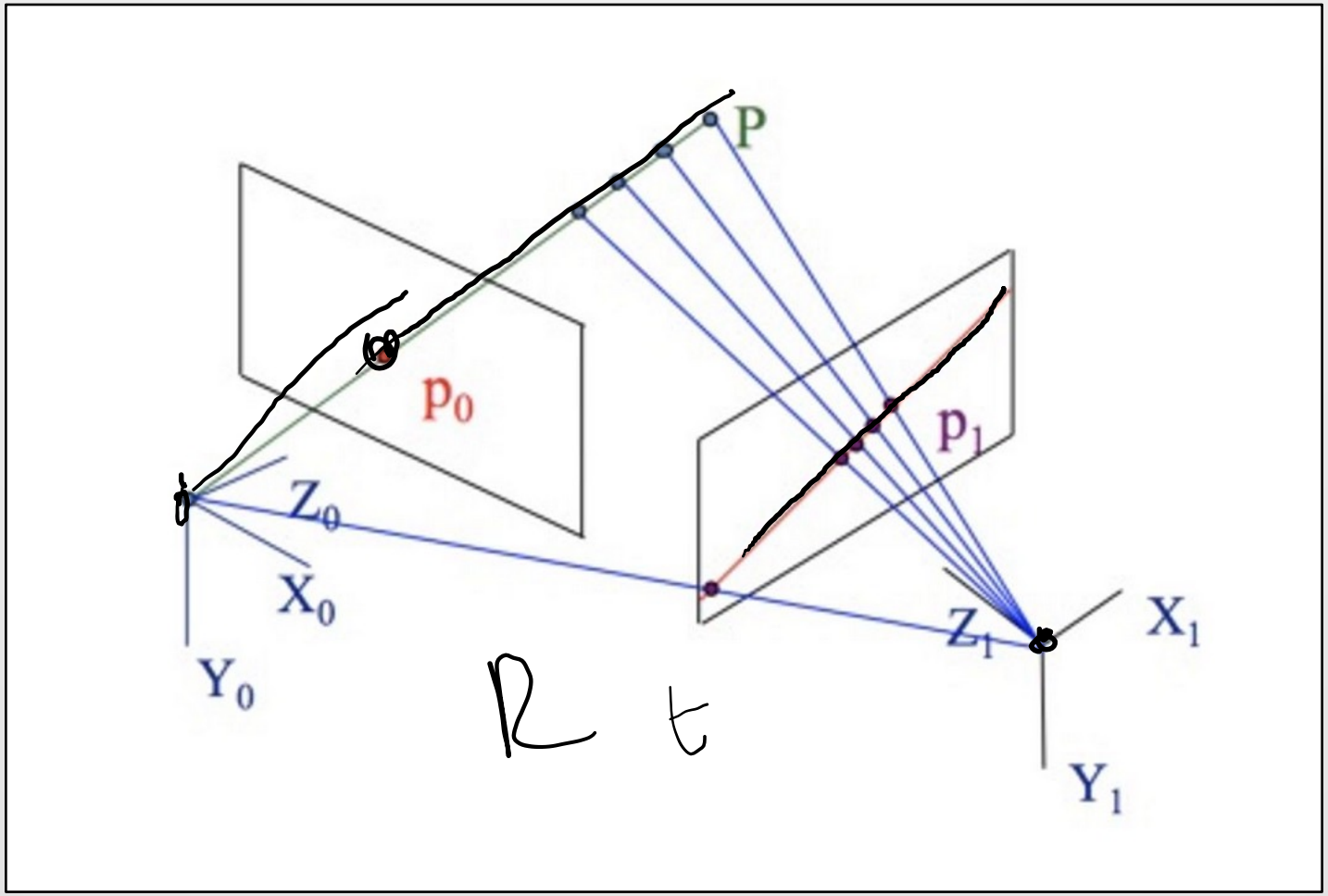


from Hartley & Zisserman

Essential Matrices

- Essential matrices encode epipolar constraints which are geometric rules which relate how projections work with cameras

Essential Matrices

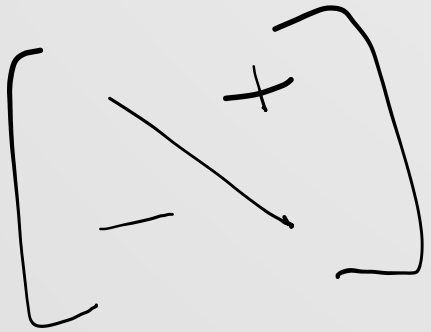


Essential Matrices

- **Essential matrices encode epipolar constraints which are geometric rules which relate how projections work with cameras**
- **In this setup,**

$$\mathbf{p}_1^\top \mathbf{E} \mathbf{p}_0 = 0$$

$$\mathbf{E} = [\mathbf{t}]_\times \mathbf{R}$$



Fundamental Matrices

- Essential matrices assume that the cameras are calibrated and the intrinsics are known
- Fundamental matrices generalize this to the case where the calibration parameters are not known
- It has the form

$$\mathbf{p}_1^\top \mathbf{F} \mathbf{p}_0 = 0$$

$$\mathbf{E} = \left(\mathbf{K}^1\right)^\top \mathbf{F} \mathbf{K}^0$$

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Geometric Verification

- The output is the geometrically validated set of potential overlap images

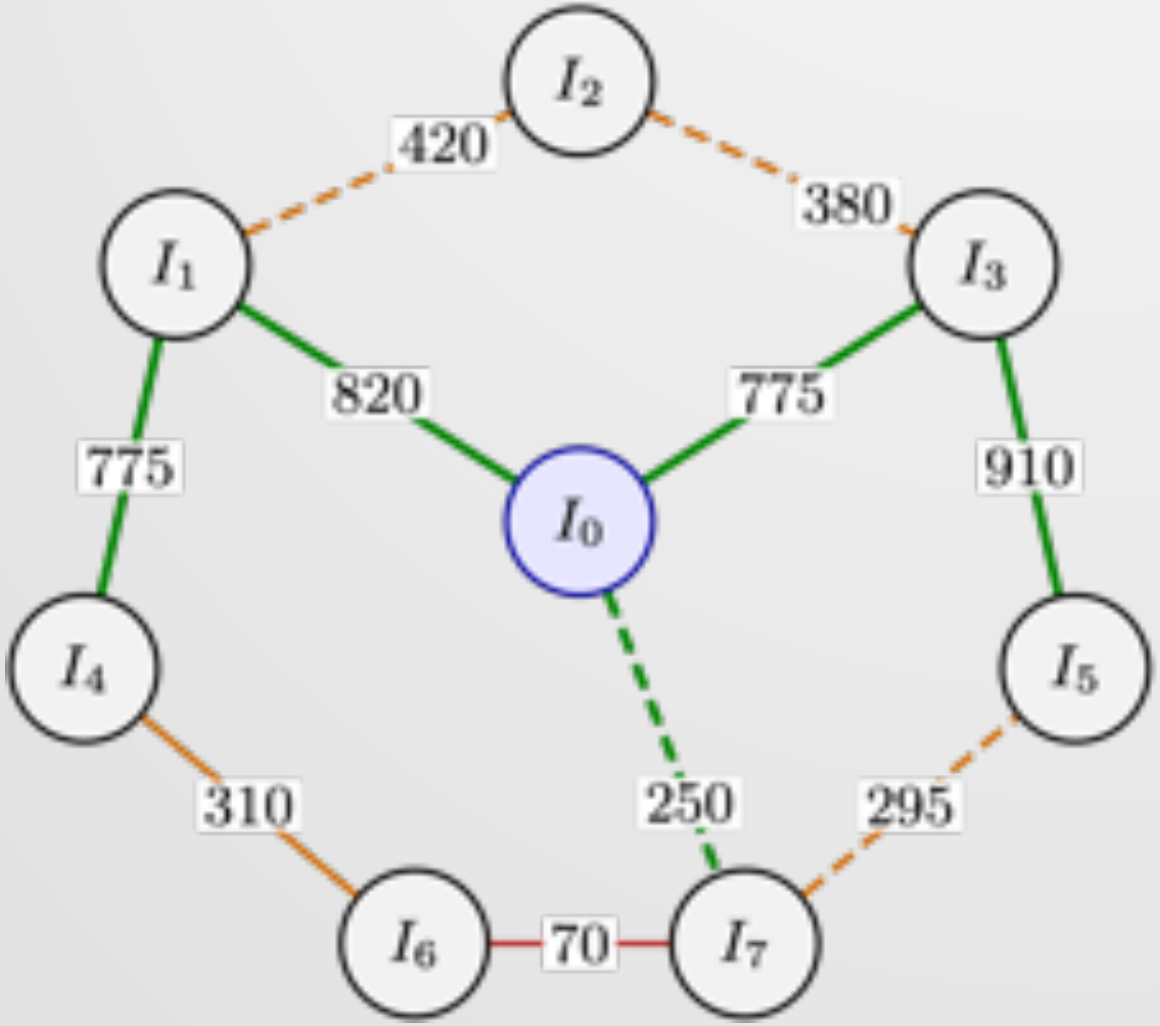
$$\bar{c}$$

- The association vector is

$$\bar{\mathcal{M}}_{ab}$$

Scene Graph

- The scenegraph stores all the two view relationships which have been determined
- The vertices are all the frames which have been matched
- The edges are a function of the type of geometric model
- The edges have a strength which is the count of the number of successful matches



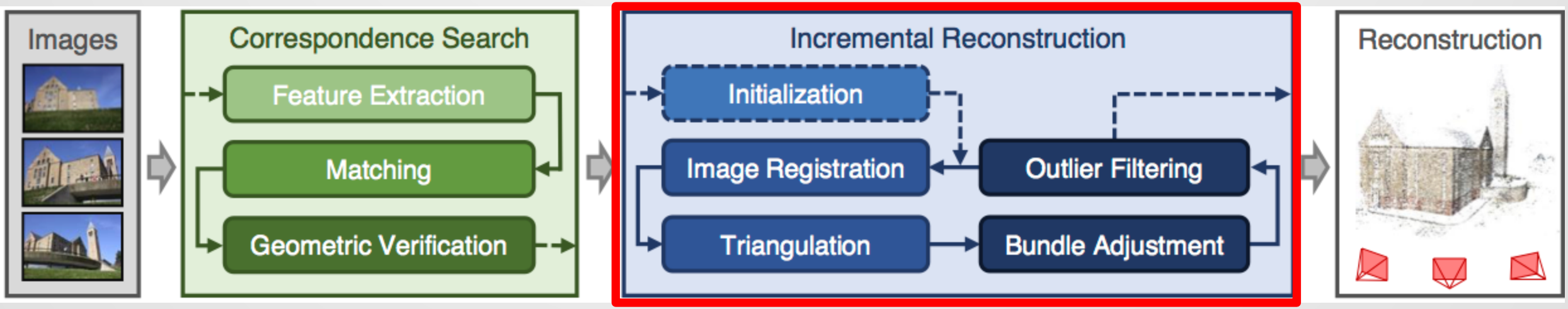
COLMAP Scene Graph
(after correspondence search)

Model (RANSAC)

——— solid: F (non-planar)

- - - - - - dashed: H (planar/rotation)

Incremental Reconstruction



[COLMAP's incremental Structure-from-Motion pipeline]

Incremental Reconstruction

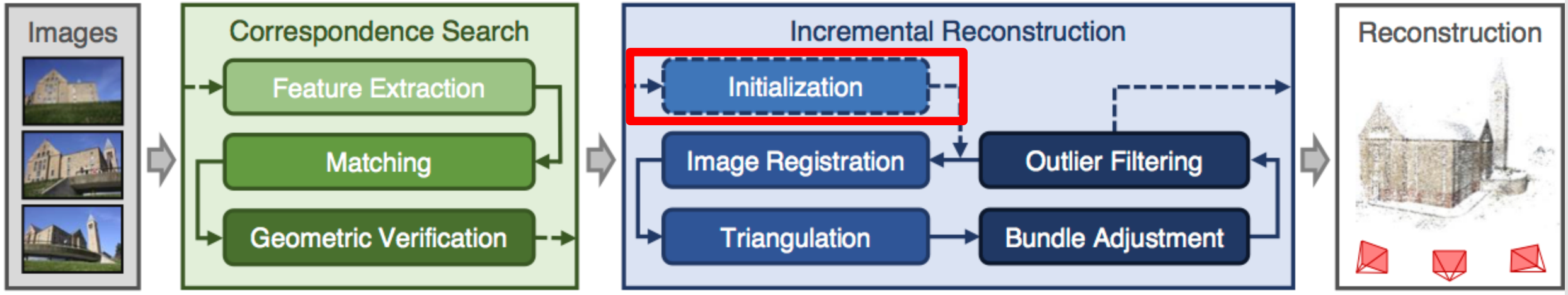
- This step takes the scenegraph and uses it (COLMAP notation!) to estimate a set of camera:

$$\mathcal{P} = \{\mathbf{P}_c \in \mathbf{SE(3)} | c = 1, \ldots, N_P\}$$

- A set of map points:

$$\mathcal{X} = \{\mathbf{X}_\ell \in \mathbb{R}^3 | \ell = 1, \ldots, N_X\}$$

Initialization



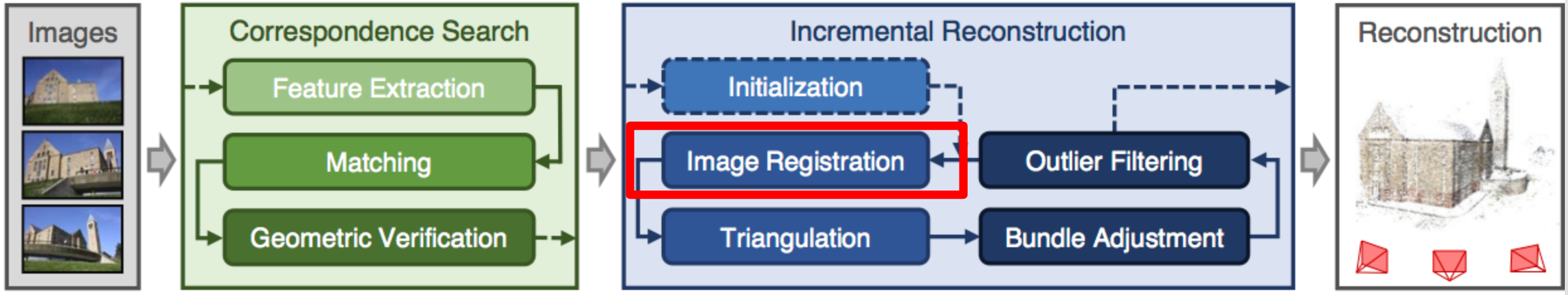
[COLMAP's incremental Structure-from-Motion pipeline]

Initialization

- COLMAP has to work out where to start from
- Picking the right point is critical to get good results
- COLMAP starts with locations in the scenegraph with the mostly highly weighted edges
- These (probably) contain the densest part of the set with the most initial overlap

Initialization

Image Registration



[COLMAP's incremental Structure-from-Motion pipeline]

Image Registration

- In this step, we take a frame which hasn't been incorporated in the 3D model and try to fit it into our growing model

- This requires next best view selection:
  - For each image that hasn't been registered yet, score it
  - Add the highest scoring image

Scoring and Next Best View

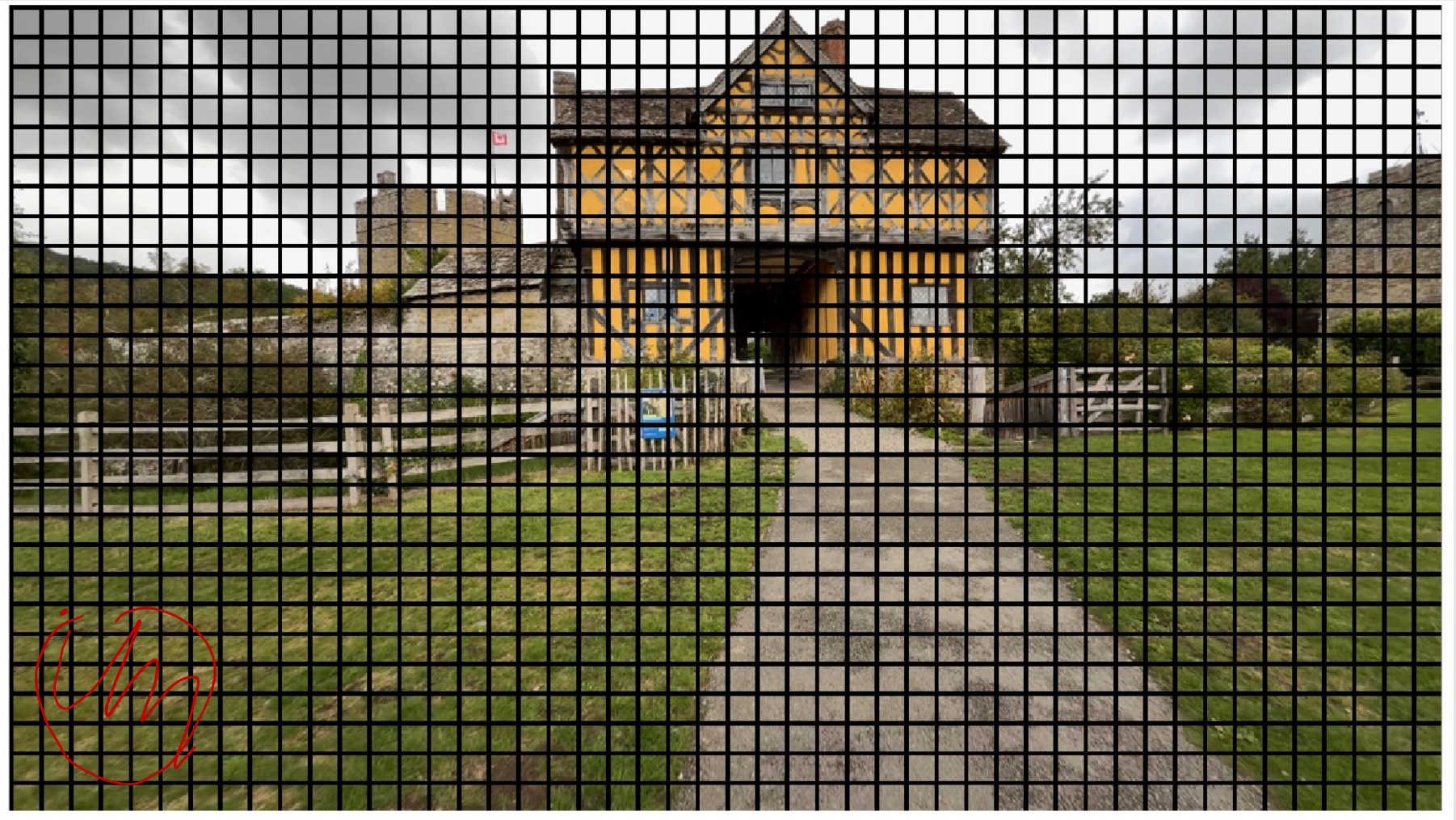
- One simple is to insert a new image with the highest overlap with the existing features in the map
- Therefore, the scenegraph can be checked and the image with the highest weight is incorporated
- Although this maximizes the number of matches, it doesn't take account of the *spatial distribution of pixels*

Spatial Distribution of Features

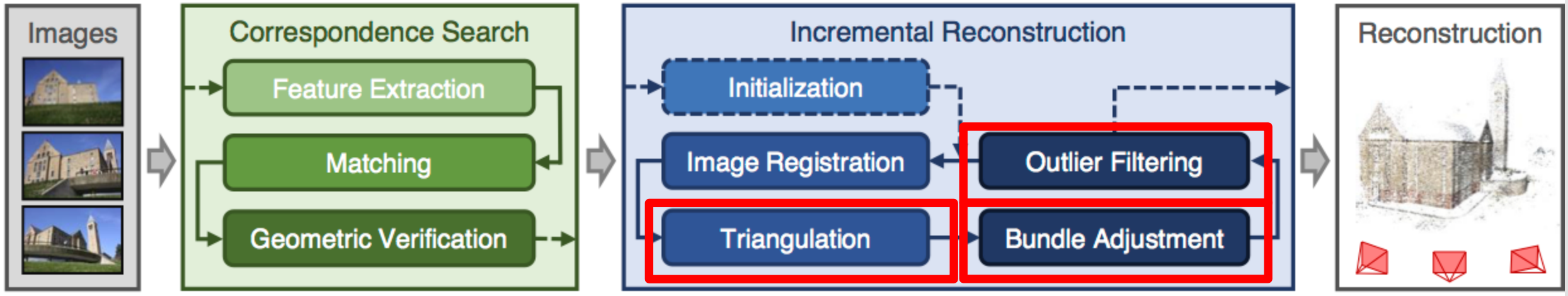
Next Best View Selection

- Each image is broken up into a set of grid cells
- The number of cells containing at least one feature are counted
- Higher counts imply a more uniform distribution of points over the image
- Multiple octaves can be used

Measuring the Spatial Distribution of Pixels



Triangulation, Outlier Filtering and Bundle Adjustment



[COLMAP's incremental Structure-from-Motion pipeline]

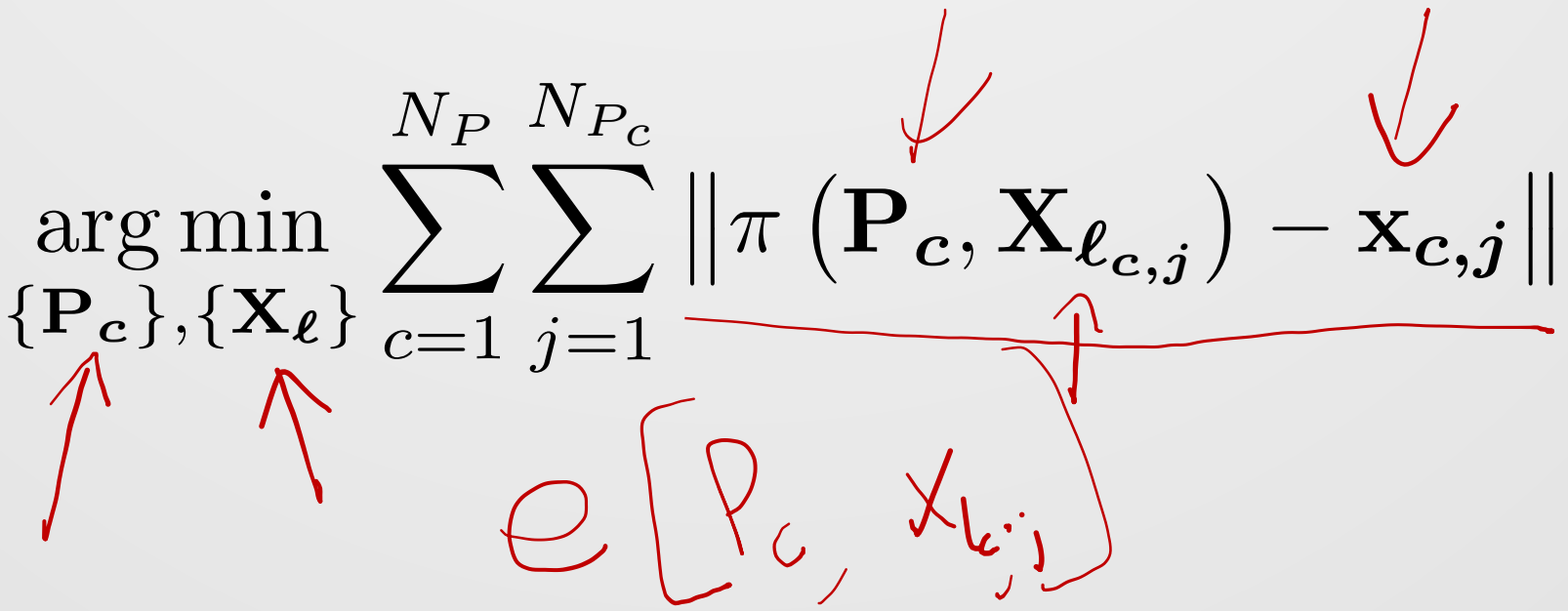
Triangulation, Outlier Filtering and Bundle Adjustment

- These steps are highly coupled with one another and can't be separated
- The output of these steps is the reconstruction, consisting of the map points and camera positions
- It has to be robust to still a potentially high degree of incorrect associations
- Before we look at the pseudocode, we need to add feature tracks

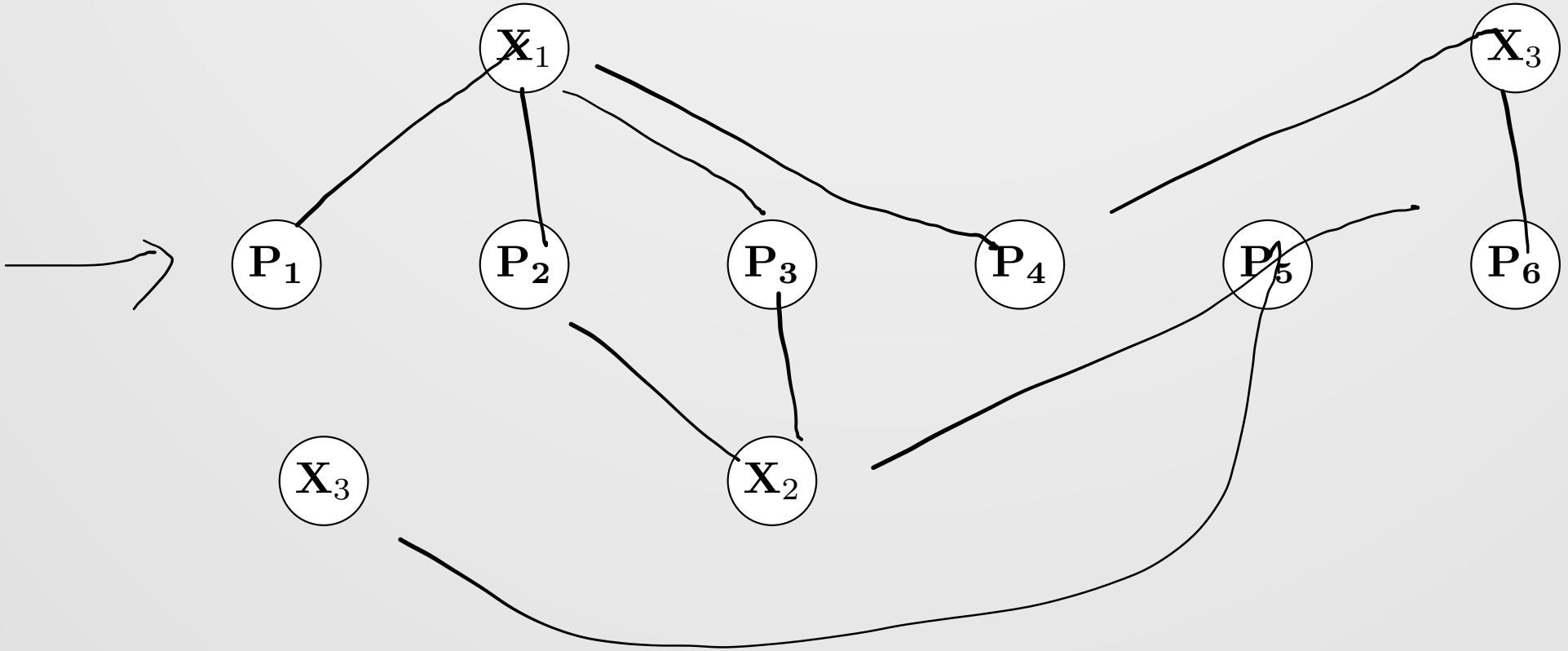
Triangulation, Outlier Filtering and Bundle Adjustment

- **The BA problem can be written as**

$$\underset{\{\mathbf{P}_c\},\{\mathbf{X}_\ell\}}{\arg\min} \sum_{c=1}^{N_P} \sum_{j=1}^{N_{P_c}} \left\| \pi\left(\mathbf{P}_c, \mathbf{X}_{\ell_{c,j}}\right) - \mathbf{x}_{c,j} \right\|^2$$



Bundle Adjustment and the Factor Graph



Feature Tracks

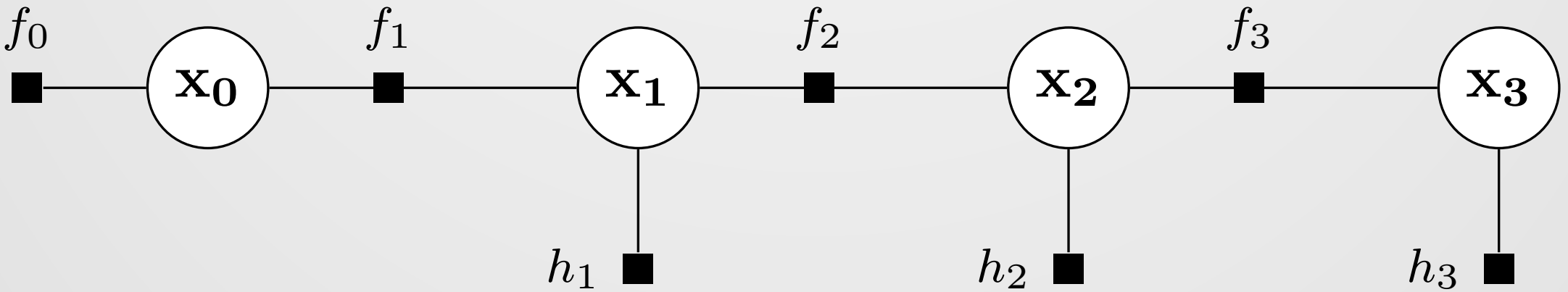
- A common grouping in SfM is to think about feature tracks
- A feature track collects all the observations of the *same point* identified over multiple frames,

$$\mathcal{T} = \{T_n | n = 1, \ldots, N_t\}$$

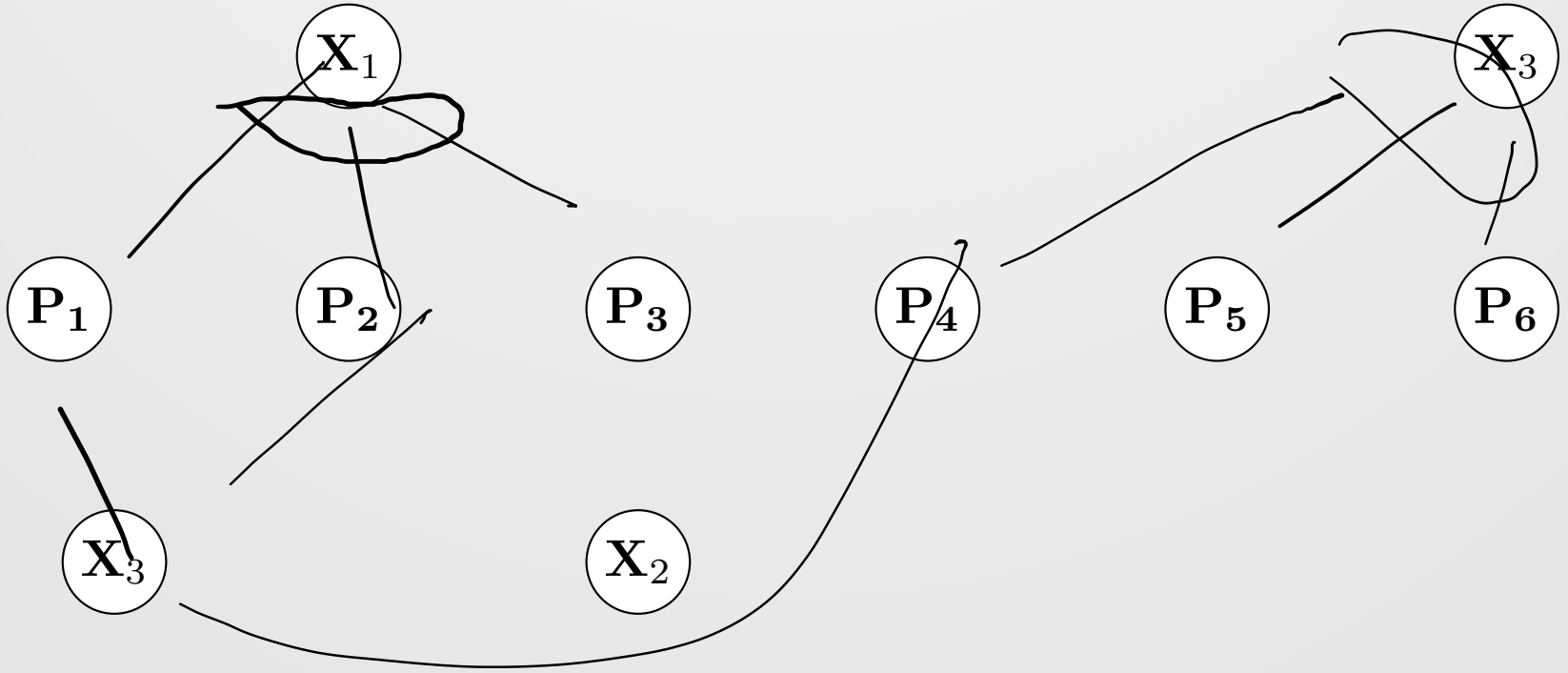
$$T_n = (\mathbf{x}_n, \mathbf{P}_n)$$

- Note that since the images are unordered, there's no sense of time ordering, dynamics, process models, etc.

Tracks vs. Feature Tracks



Tracks vs. Feature Tracks



Algorithm 1 Incremental SfM with Track Extension, Robust Triangulation, and BA

Require: Images $\mathcal{I}$, intrinsics (fixed or variable), pairwise matches
Ensure: Camera poses $\{\mathbf{P}_j\}$, 3D points $\{\mathbf{x}_\ell\}$, observation tracks

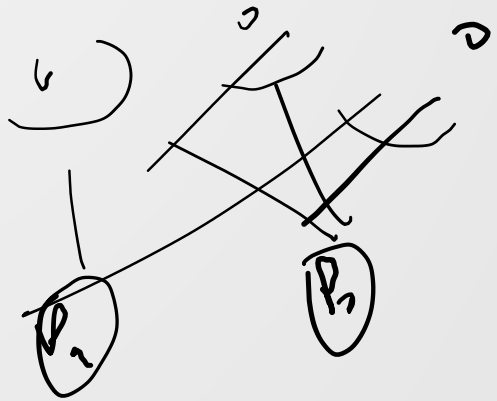
Initialize: choose seed pair (I_a, I_b)
Estimate relative pose; register cameras $\mathbf{P}_a, \mathbf{P}_b$
Form initial tracks from verified matches between (I_a, I_b)
Triangulate eligible tracks to create initial 3D points $\{\mathbf{x}_\ell\}$
BundleAdjust($\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$, obs)
FilterOutliers(obs, $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$)
while ExistsUnregisteredImage($\mathcal{I}$) **do**
    $I_{\text{new}} \leftarrow$ SelectNextImage($\mathcal{I}$, current model)
    $\mathbf{P}_{\text{new}} \leftarrow$ RegisterImagePnP($I_{\text{new}}, \{\mathbf{x}_\ell\}$, obs)
    if $\mathbf{P}_{\text{new}} = \varnothing$ **then**
        continue
    end if
    AddCamera($\mathbf{P}_{\text{new}}$)
    ▷ — Track / point update for newly registered image —
    ExtendTracks(I_{new}, matches, tracks/obs)
    TriangulateNewPoints(tracks/obs, $\{\mathbf{P}_j\}$)
    ▷ — Inner refinement loop —
    repeat
        BundleAdjust($\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$, obs)
        $n_{\text{removed}} \leftarrow$ FilterOutliers(obs, $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$)
        PrunePoints($\{\mathbf{x}_\ell\}$, obs)
        RetriangulateIfNeeded(tracks/obs, $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$)
    until $n_{\text{removed}} = 0$
end while
▷ Optional global cleanup
BundleAdjust($\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$, obs)
FilterOutliers(obs, $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$)
return $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$, obs/tracks

Algorithm 1 Incremental SfM with Track Extension, Robust Triangulation, and BA

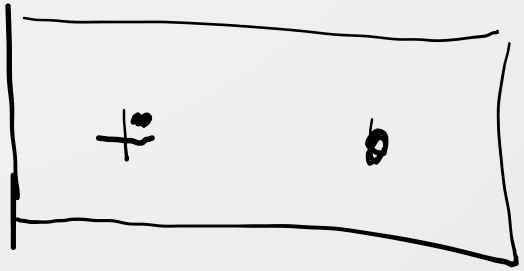
Require: Images $\mathcal{I}$, intrinsics (fixed or variable), pairwise matches

Ensure: Camera poses $\{\mathbf{P}_j\}$, 3D points $\{\mathbf{x}_\ell\}$, observation tracks

1: **Initialize:** choose seed pair (I_a, I_b)
2: Estimate relative pose; register cameras $\mathbf{P}_a, \mathbf{P}_b$
3: Form initial tracks from verified matches between (I_a, I_b)
4: Triangulate eligible tracks to create initial 3D points $\{\mathbf{x}_\ell\}$
5: BundleAdjust($\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$, obs)
6: FilterOutliers(obs, $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$)



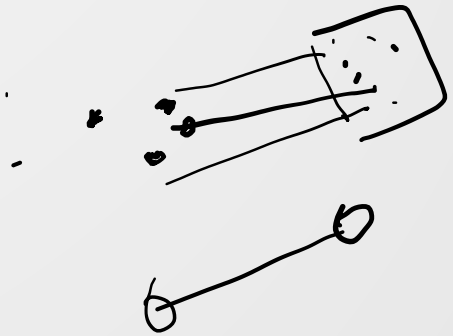
Filtering Outliers



- **Outliers are determined by computing a threshold on the projection error,**

$$\begin{cases} \text{inliner} & \left\| \pi\left(\mathbf{P}_c, \mathbf{X}_{\ell_{c,j}}\right) - \mathbf{x}_{c,j} \right\|^2 < t \\ \text{outlier} & \text{otherwise} \end{cases}$$

```
while ExistsUnregisteredImage(I) do
    I_new ← SelectNextImage(I, current model)
    P_new ← RegisterImagePnP(I_new, {x_ℓ}, obs)
    if P_new = ∅ then
        continue
    end if
    AddCamera(P_new)
                ▷ — Track / point update for newly registered image —
    ExtendTracks(I_new, matches, tracks/obs)
    TriangulateNewPoints(tracks/obs, {P_j})
                                ▷ — Inner refinement loop —
    repeat
        BundleAdjust({P_j}, {x_ℓ}, obs)
        n_removed ← FilterOutliers(obs, {P_j}, {x_ℓ})
        PrunePoints({x_ℓ}, obs)
        RetriangulateIfNeeded(tracks/obs, {P_j}, {x_ℓ})
    until n_removed = 0
end while
```



23: ▷ Optional global cleanup

24: BundleAdjust($\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$, obs)

25: FilterOutliers(obs, $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$)

26: **return** $\{\mathbf{P}_j\}, \{\mathbf{x}_\ell\}$, obs/tracks

---

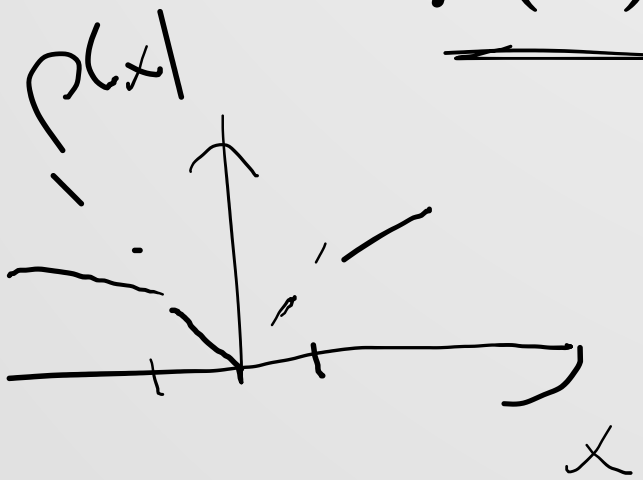
Robust Kernels

- The pseudocode includes a lot of checks for errors
- However, all the inlier and outlier detection is based on ourrent estimates
- If these are wrong, then the BA computed solution is wrong and the inlier / outlier associations are wrong
- Therefore, a robust kernel is used

Robust Kernels

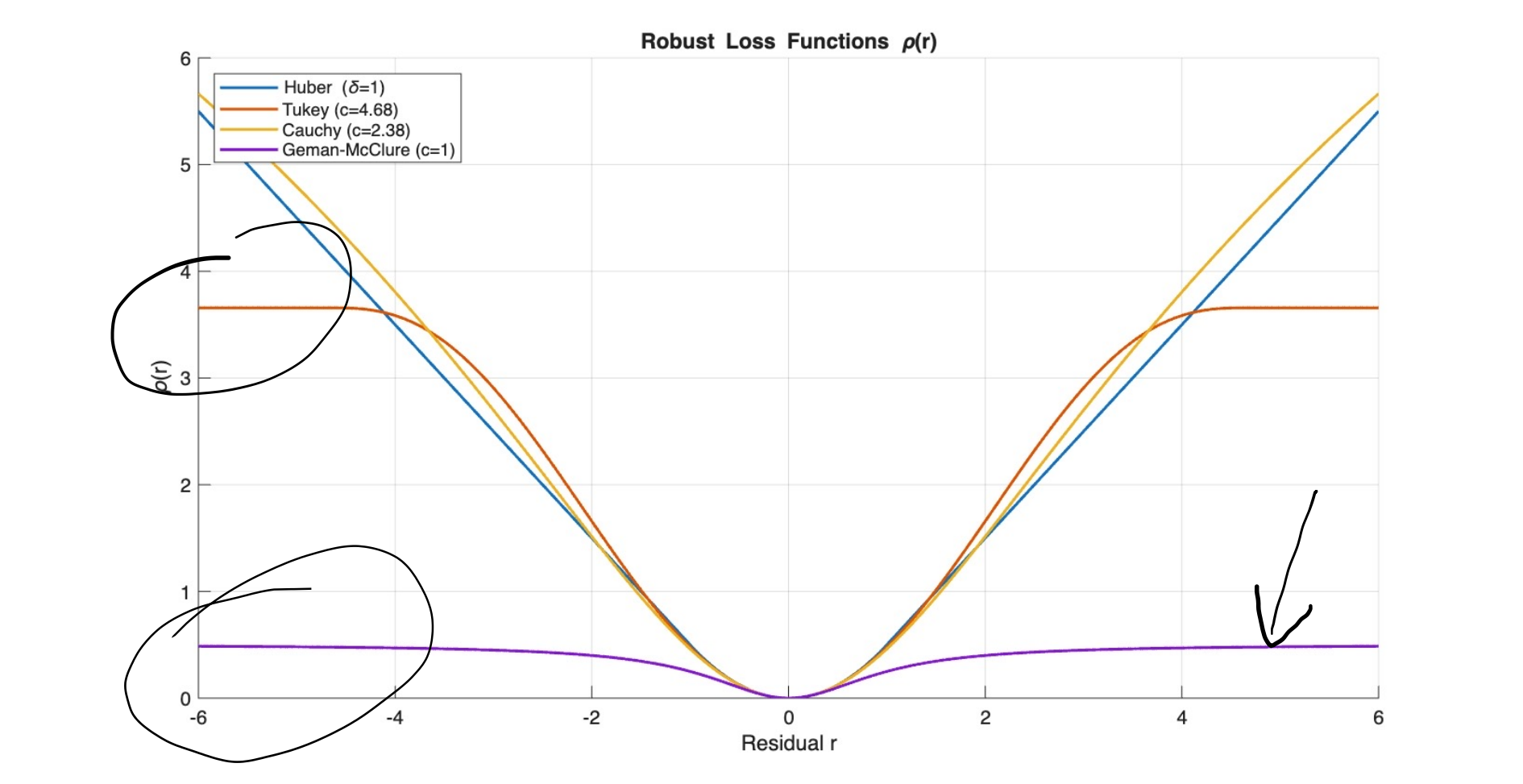
- A robust kernel looks roughly like

$$\rho(x) = \begin{cases} x & |x| \leq t \\ \sigma(x) & \text{otherwise} \end{cases}$$



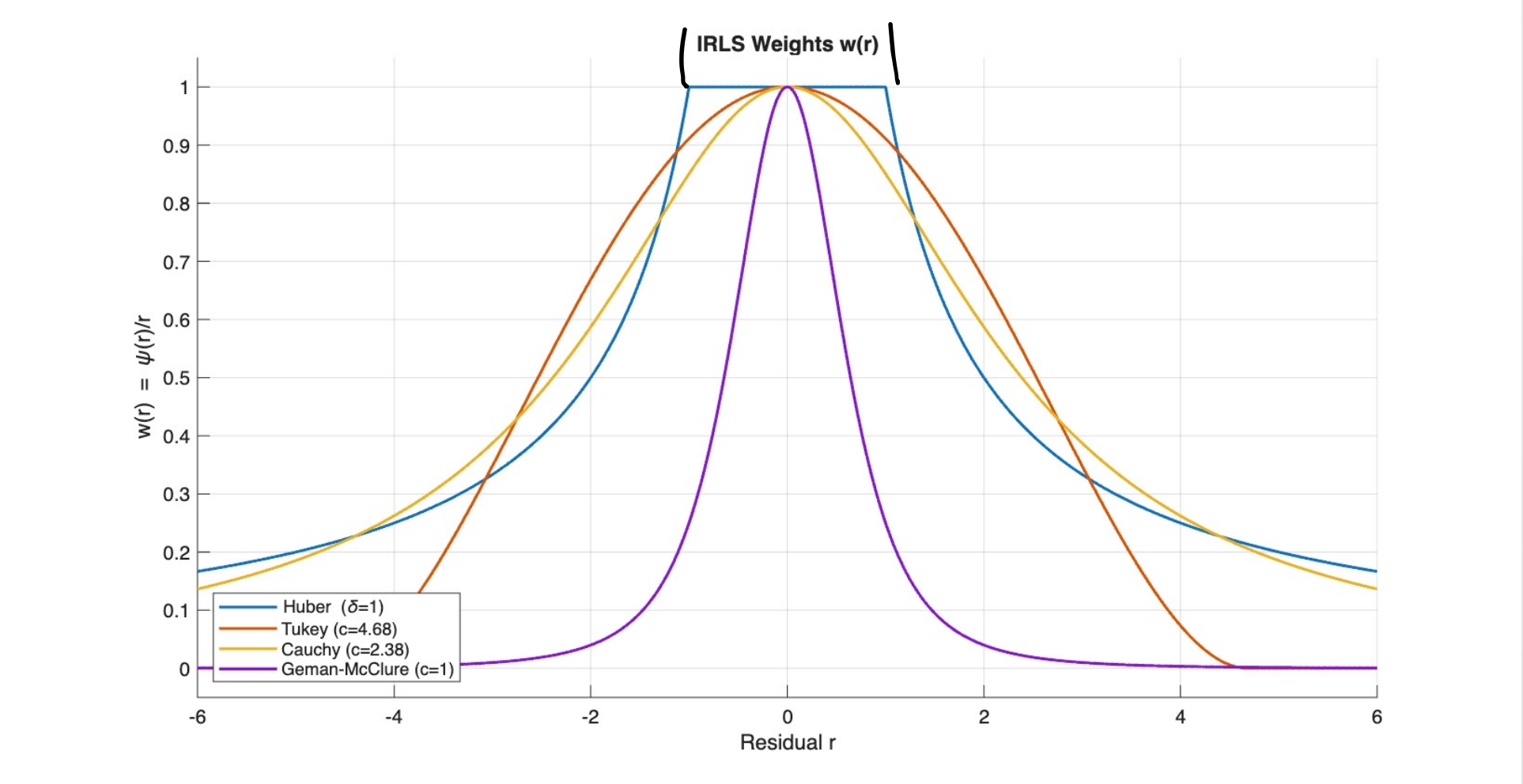
$$|\sigma(x)| \leq |x|$$

Common Robust Kernels



Common Robust Kernels

$$\frac{|\sigma(x)|}{|x|}$$



Robust BA

$$\underset{\{\mathbf{P_c}\},\{\mathbf{X}_{\boldsymbol{\ell}}\}}{\arg\min} \sum_{c=1}^{N_P} \sum_{j=1}^{N_{P_c}} \boldsymbol{\rho}\left(\left\|\pi\left(\mathbf{P}_{\boldsymbol{c}}, \mathbf{X}_{\boldsymbol{\ell}_{\boldsymbol{c},\boldsymbol{j}}}\right) - \mathbf{x}_{\boldsymbol{c},\boldsymbol{j}}\right\|\right)^2$$

Examples

- **Hopefully we'll try something live(ish)**

Summary

- COLMAP is a widely used SfM system
- In this shallowish dive, we've seen that to move from an estimation problem to a full operational system requires a lot of work
- We've had some hints of how SLAM systems work
- However, when we look at ORB-SLAM2, we'll see how it adapts these ideas to produce a system that's even more flexible